TUTORIAL DE GIT · CAPÍTULO 7

Herramientas de Git

Selección de revisiones, stash, reescritura de historia, reset, depuración y submódulos

1. Selección de revisiones

Git permite referirse a un commit de varias formas:

Forma	Ejemplo y significado
Hash corto	`git show 1c002dd` (basta con los primeros caracteres si no hay ambigüedad)
Referencias de rama	`git show main`
Ancestros	`HEAD~2` (dos commits antes de HEAD) y `HEAD^` (el padre)
Rangos (doble punto)	`git log main..prueba` (commits en prueba que no están en main)
Reflog	`git reflog` y `HEAD@{2}` (dónde estuvo HEAD hace dos movimientos)

2. Preparación interactiva

```
git add -i      # menú interactivo para preparar archivos
git add -p      # prepara trozos (hunks) de un archivo, no el archivo entero
```

3. Guardado rápido (stash)

`git stash` guarda temporalmente los cambios sin confirmar para dejar el directorio de trabajo limpio, por ejemplo para cambiar de rama.

```
git stash                 # guardar cambios
git stash list            # ver lo guardado
git stash apply           # recuperar los cambios (se conservan en la lista)
git stash pop             # recuperar y eliminar de la lista
git stash drop            # eliminar el último guardado
git clean -f -d           # borrar archivos sin seguimiento (¡irreversible!)
```

4. Búsqueda e historial

```
git grep -n "texto"         # buscar texto en los archivos del proyecto
git log -S"funcion"         # commits que añadieron o quitaron esa cadena
git log -L :funcion:archivo.py   # historia de una función concreta
git blame archivo.txt       # quién modificó cada línea y en qué commit
git bisect start            # búsqueda binaria del commit que introdujo un
error
git bisect bad
git bisect good v1.0
```

5. Reescribir la historia

```
git commit --amend            # modificar el último commit
git rebase -i HEAD~3          # editar interactivamente los 3 últimos commits
```

En el rebase interactivo se puede **reordenar** commits, **eliminarlos**, **editar** el mensaje (`reword`) o **combinarlos** en uno solo (`squash`). Existen herramientas más potentes como `git filter-branch` para reescribir muchos commits a la vez.

> *Solo se debe reescribir la historia de commits que no se han publicado.*

6. Reset desmitificado

`git reset` mueve la rama actual (HEAD) a otro commit y, según la opción, afecta también al área de preparación y al directorio de trabajo:

Opción	Efecto
`--soft`	Mueve solo HEAD; los cambios quedan preparados.
`--mixed` (por defecto)	Mueve HEAD y vacía el área de preparación; los cambios quedan en el directorio de trabajo.
`--hard`	Mueve HEAD y restablece también el directorio de trabajo; **se pierden los cambios no confirmados**.

```
git reset --soft HEAD~1    # deshacer el último commit conservando los cambios preparados
git reset --hard HEAD~1    # deshacer el último commit y descartar los cambios
```

7. Otras herramientas

- **Firmar el trabajo con GPG:** `git commit -S` y `git tag -s` permiten verificar la autoría.
- **Fusión avanzada:** `git merge --abort` cancela una fusión; `git rerere` recuerda cómo se resolvió un conflicto para reutilizarlo.
- **Submódulos:** incluyen otro repositorio dentro del proyecto (`git submodule add <url>`; al clonar: `git clone --recurse-submodules` o `git submodule update --init --recursive`).
- **Empaquetado:** `git bundle create repo.bundle main` guarda un repositorio en un solo archivo transportable.
- **Almacenamiento de credenciales:** `git config --global credential.helper cache` evita escribir la contraseña repetidamente.

> *Resumen: estas herramientas son avanzadas pero muy útiles: stash para guardar trabajo a medias, rebase interactivo para limpiar la historia, reset para mover ramas y bisect para encontrar errores.*

Fuente: Adaptado de Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2 (licencia CC BY-NC-SA 3.0).